

Kubernetes Labs

Project Summary | Local Minikube Cluster | Merter Akinci

Status: Core concepts completed | Built entirely on a free, local Minikube cluster (Docker driver, Calico CNI) — no cloud costs incurred. Learning path targets the CKA and CKS certifications.

1. Project Overview

This project is a hands-on Kubernetes learning path, covering core workload management, configuration, security (RBAC and NetworkPolicy), and storage concepts. Each topic was implemented as declarative YAML, applied to a local cluster, and verified with concrete tests rather than assumed to work. It follows on from a separate Terraform/AWS project (ecommerce-infra-as-code), continuing the same infrastructure-as-code mindset in the container orchestration space.

Repository: github.com/merterakinci/k8s-labs

2. Environment

Component	Choice	Reason
Cluster	Minikube (local)	Free, matches CKA/CKS exam-style practice without cloud cost
Driver	Docker	VirtualBox unsupported on Apple Silicon (darwin/arm64)
CNI	Calico	Required for NetworkPolicy enforcement (default bridge CNI does not support it)
CLI	kubectl	Standard Kubernetes command-line tool

3. Repository Structure

```
k8s-labs/  
■■■■ 01-first-pod/           # Pod + Service basics (imperative -> declarative)  
■■■■ 02-deployment/         # Deployment: replicas, self-healing, rolling updates  
■■■■ 03-configmap-secret/    # ConfigMap and Secret injected as env variables  
■■■■ 04-rbac/                # ServiceAccount, Role, RoleBinding (least privilege)  
■■■■ 05-network-policy/      # NetworkPolicy, verified with a test client pod  
■■■■ 06-persistent-storage/  # PersistentVolumeClaim, verified across pod deletion  
■■■■ README.md
```

4. What Was Built and Verified

Topic	What Was Done	Verification Method
Pod & Service	Deployed nginx via YAML, exposed via NodePort	Accessed nginx welcome page in browser via minikube tunnel
Deployment	3 replicas, self-healing, rolling update to nginx:1.25.5	Manually deleted a pod and observed automatic recreation; watched

ConfigMap / Secret	Injected config and a mock API key as env variables	kubectl exec + env, confirmed values present inside the container
RBAC	Read-only ServiceAccount, Role limited to get/list/watch on pods	kubectl auth can-i checks: allowed to list, denied to delete/create
NetworkPolicy	Restricted ingress to nginx pods to same-label pods only	Tested client could reach nginx before the policy, timed out after applying
Persistent Storage	PVC mounted into a pod at /data	Wrote a file, deleted the pod, recreated it, confirmed the file still existed

5. Skills Demonstrated

- Declarative infrastructure management with Kubernetes YAML manifests (mirrors the Terraform IaC approach used in the AWS project)
- Core workload primitives: Pod, Deployment, ReplicaSet, Service (NodePort)
- Configuration management: ConfigMap and Secret injection via environment variables
- Security: RBAC least-privilege design (ServiceAccount, Role, RoleBinding), verified with kubectl auth can-i
- Network segmentation: NetworkPolicy enforced via Calico CNI, verified with a live before/after test
- Storage: PersistentVolumeClaim, verified that data survives pod deletion and recreation
- Practical troubleshooting: resolved a driver incompatibility (VirtualBox on Apple Silicon), a Docker daemon not running error, and an engine version mismatch

6. Why Minikube Instead of a Cloud Cluster (EKS)

A managed Kubernetes service (e.g. AWS EKS) was intentionally avoided for this learning phase due to ongoing cost (control plane + worker node charges, not covered by the AWS Free Tier). Minikube provides an equivalent hands-on experience for CKA/CKS exam preparation at zero cost, since the certification exams themselves are also performed against a provided cluster via kubectl, independent of the underlying infrastructure.

7. Next Steps

- Namespaces and resource quotas
- StatefulSets (for stateful applications like databases)
- Ingress controllers (HTTP routing, replacing NodePort for production-style access)
- Helm (package management for Kubernetes manifests)
- CKA practice exams (killer.sh) and formal certification attempt
- CKS-specific topics: Pod Security Standards, image scanning, runtime security (Falco)